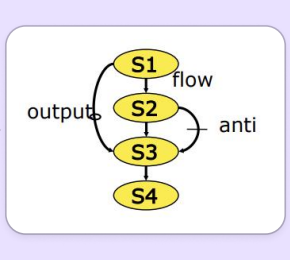
dependencies and parallel analysis

dependencies and parallel analysis

- **parallelism and correctness**
  - parallel execution is fundamentally constrained by the sequence of operations required to produce a correct result
  - to execute a program in parallel, we must address three types of dependencies
    - control
    - data
    - system dependencies
  - a **dependencies** arises when a specific operation relies on the completation of an earlier operation to produce a result before it can proceed
    - this concept extends to resources as well, such as data location
  - when executing two statements on multiple processors, the fundamental assumption of concurrent execution is that processors execute independently of one another
    - no assumptions made regarding their relative execution speeds
- **sequential consistency**
  - the gold standard for **correctness** in parallel computing is **sequential consistency**
    - the execution of statements must not interfere with one another
    - the computation results must remain the same, independent of the order of execution
  - in other words, the execution of $S1;S2;$ must be equivalent to the execution of $S2;S1;$
    - if this holds true, the statements are **independent**
    - if the order affects the outcome, the statements are **dependent**

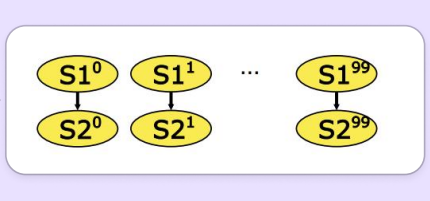
  SEQUENTIAL CONSISTENCY = the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program

- **data dependencies**
  - **classifying data dependencies**
    - statement S1 and S2 can execute in parallel if and only if there are no dependences between S1 and S2
    - dependencies are categorized based on how statements read (IN sets) and write (OUT sets) memory locations
      - **true (flow) dependence (δ)**
        - dipendenza "reale" perché i dati fluiscono fisicamente da un'istruzione all'altra → Read-After-Write (RAW)
        - statement S2 has a flow dependence on S1 *if and only if* S2 reads a value written by S1
          - S1: a=1
S2: b=a
            - S1 scrive un valore che S2 legge successivamente
            - la dipendenza non può essere rimossa senza cambiare l'algoritmo
        - **logic:** S1 produces data that S2 consumes (X=... then ...=X)
          - the second is dependent on first
      - **anti-dependence (δ^{-1})**
        - Write-After-Read (WAR)
        - statement S2 has an anti-dependence on S1 *if and only if* S2 writes a value that is read by S1
          - S1: a=b
S2: b=1
            - S1 legge un valore che S2 successivamente sovrascrive → se invertissimo l'ordine (eseguendo S2 prima di S1), S1 leggerebbe il valore sbagliato
        - **logic:** S1 must read the old value before S2 overwrites it (...=X them X=...)
          - the first is dependent on second
        - **solution:** unlike true dependencies, it can often be removed by modifying the program
          - **ex.** renaming variables or eliminating statements
          - it can be called **"name" dependencies**
            - può essere tolta rinominando la variabile o usando memoria locale
      - **output dependence (δ^{0})**
        - Write-After-Write (WAW)
        - statement S2 has an output dependence on S1 *if and only if* S2 writes to a variable that is also written by S1
          - S1: a=f(x)
S2: a=b
            - sia S1 che S2 scrivono nella stessa variabile → ordine importante poiché determina quale valore finale rimane in memoria
        - **logic:** the final value of the variable depends on which statement writes last (X=... then X=...)
          - the second is dependent on first
        - **solution:** unlike true dependencies, it can often be removed by modifying the program
          - **ex.** renaming variables or eliminating statements
          - it can be called **"name" dependencies**
  - **dependency analysis techniques**
    - to systematically analyze dependencies, we definte **IN** and **OUT** sets for a statement S
      - **IN(S):** the set of memory locations (variables) that may be used (read) in S
      - **OUT(S):** the set of memory locations that may be modified (written) by S
      - per verificare se esiste una dipendenza tra S1 (prima) e S2 (dopo), analizziamo le intersezioni di questi insiemi
    - assuming that there is a path from S1 to S2, the following shows how to intersect the IN and OUT sets to test for data dependence
      - **true (flow) dependence**
        - **OUT(S1) (∩) IN(S2) = ø**
          - S1 scrive qualcosa che S2 legge
      - **anti dependence**
        - **IN(S1) (∩) OU(S2) = ø**
          - S1 legge qualcosa che S2 sovrascrive
      - **output dependence**
        - **OUT(S1) (∩) OUT(S2) = ø**
          - entrambi scrivono nella stessa locazione
    - we can visualize these relationships using **dependency graphs** where nodes rapresent statements and directed edges represent the type of dependence connecting them
      - statements can be execute in parallel *if and only if* there are no dependencies between them
      - **example**
        - S1: a=1
S2: b=a
S3: a=b+1
S4: c=a
        - 

          - **S2 δ S3:** S3 is flow-dependent on S2
          - **S1 δ^(0) S3:** S3 is output-dependent on S1
          - **S2 δ^(-1) S3:** S3 is anti-dependent on S2

- **loop-level parallelism**
  - **loop-independent dependencies**
    - la dipendenza esiste all'interno della stessa iterazione (quindi tra due righe di codice dentro lo stesso for)
    - a loop is a **DOALL loop** (or foreach) if all iterations are independent
      - all statements in all iterations can be executed in parallel simultaneously
        - poiché la dipendenza è confinata all'interno della singola iterazione, ogni iterazione i rimane indipendente dall'iterazione j
      - we can verify this by "unrolling" the loop to visualize dependencies between separate iteratins

        ```
        for (i=0; i<100; i++)
          S1: a[i]=i;
          S2: b[i]=2*i;
        ```

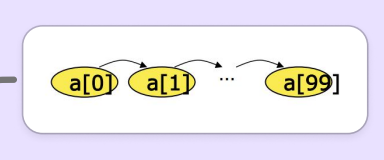
        - S2 deve aspettare S1, ma l'iterazione i=1 non deve aspettare l'iterazione i=0
        - 

          - we only care about knowing in which range i belongs
      - we can identify significant parallelism between loops
        - consider two sequential for loops with the first that initializes array A and the second initializes array B

          ```
          for (i=0; i<100; i++)
            a[i] = i;
          for (i=0; i<100; i++)
            b[i] = i;
          ```

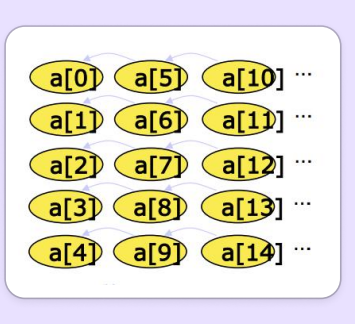
        - if the memory spaces of A and B do not overlap, these two loops are completely independent tasks
        - this scenario offers massive parallel potential
          - **task parallelism:** you can assign the entire first loop to Processor 1 and the entire second loop to Processor 2
          - **data parallelism:** within each loop, the iterations are also independent, meaning you can split the 100 iterations of A across multiple cores *while* simultaneously splitting the 100 iterations of B across others
        - when loops touch disjoint data, the parallelism is limited only by the hardware resources
          - **ex.** with 50 processors, you could theoretically process half tha data instantly
  - **loop-carried dependencies**
    - la dipendenza attraversa le interazioni (per cui l'iterazione i ha bisogno di dati prodotti dall'iterazione i-1) → impedisce la parallelizzazione totale
    - when we look *inside* a single loop, we must define how data moves between iterations
      - two critical dependency types
        - **case 1: strict flow dependence (the "serial chain")**
          - catena di dipendenza "Read-After-Write" (RAW) immediata tra un'iterazione e la precedente

            ```
            for (i=1; i<100; i++)
              a[i] = a[i-1] + 100;
            ```

          - in this case, iteration i reads the value written by the immediate previous iteration i-1
            - 

            - **heavy dependencies**
          - this loop is not parallelizable
            - you cannot calculate a[10] until a[9] is finished
            - the dependency chain forces a strict sequential execution order
        - **case 2: dependence with distance (the "partial case")**
          - mostra una dipendenza che non avviene con l'iterazione immediatamente precedente ma ad una certa "distanza" k (offset)

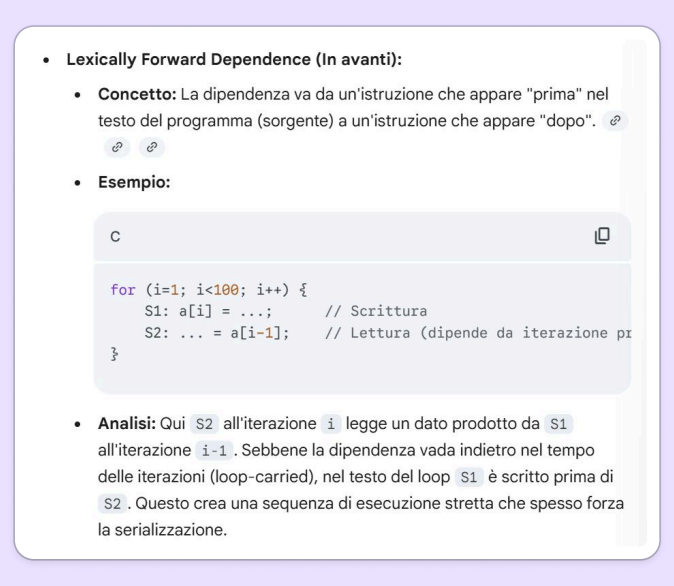
            ```
            for (i=5; i<100; i++)
              a[i-5] = a[i] + 100;
            ```

            - esiste una "Write-After-Read" in quanto l'iterazione 10 scriverà su a[5] scrivendo il valore che l'iterazione 5 deve leggere → se l'iterazione 10 avviene prima della 5 leggerebbe il valore sbagliato
          - iteration i writes to a location that was relavant 5 iterations ago (i-5)
            - conversely, iteration i reads a[i] and iteration i+5 will write to a[i]
            - this is a dependency with a **distance vector of 5**
              - 

          - this is **partially parallelizable**
            - because the dependency isn't immediate (it skips 5 steps), you don't have to wait for the *immediately* preceding iteration
            - you can execute batches of 5 iterations in parallel or use a "wavefront" execution strategy, as long as you respect the distance constraint
            - le iterazioni all'interno di una finestra di ampiezza (in questo caso di 5) sono indipendenti tra di loro e possono essere eseguite in parallelo, purché si rispetti l'ordine tra i blocchi
    - a dependence is **loop-carried** if it exists between statements in two *different* iterations of a loop
      - if a dependence is within the same iteration, it is a **loop-independent**
      - loop-carried dependencies generally prevent full parallelization of the loop

        ```
        for (i=5; i<100; i++)
          a[i] = f(a[i-1])
        ```

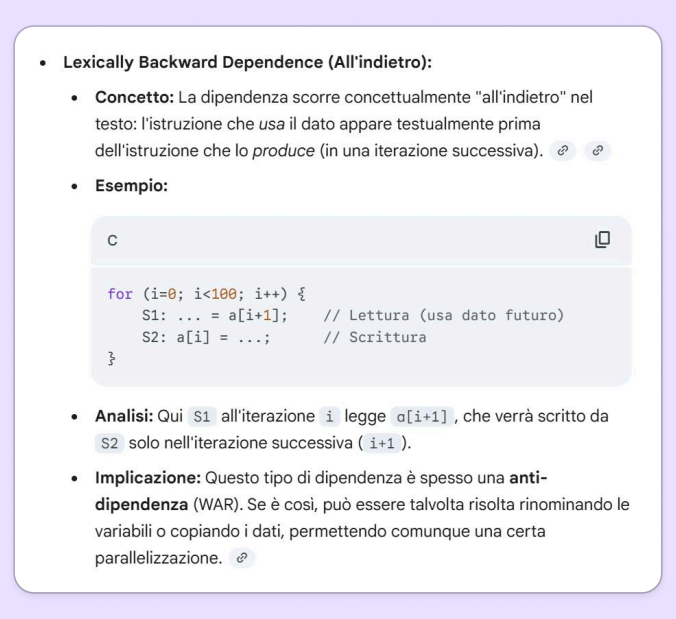
        - the calculation of a[i] depends of the result of the previous iteration a[i-1]
        - this creates a chain of dependencies that forces serial execution
    - we can distinguish between
      - distinzione che si riferisce alla posizione testuale (lessicale) delle istruzioni nel codice sorgente e alla direzione del flusso della dipendenza rispetto a questo ordine
      - **lexically forward**
        - the source of the dependence comes *before* the target in the code
      - **lexically backward**
        - the source comes *after* the target

      - **Lexically Forward Dependence (In avanti):**
        - **Concetto:** La dipendenza va da un'istruzione che appare "prima" nel testo del programma (sorgente) a un'istruzione che appare "dopo".
        - **Esempio:**

          ```
          C
          for (i=1; i<100; i++) {
              S1: a[i] = ...;      // Scrittura
              S2: ... = a[i-1];    // Lettura (dipende da iterazione pr
          }
          ```

        - **Analisi:** Qui `S2` all'iterazione `i` legge un dato prodotto da `S1` all'iterazione `i-1`. Sebbene la dipendenza vada indietro nel tempo delle iterazioni (loop-carried), nel testo del loop `S1` è scritto prima di `S2`. Questo crea una sequenza di esecuzione stretta che spesso forza la serializzazione.

      - **Lexically Backward Dependence (All'indietro):**
        - **Concetto:** La dipendenza scorre concettualmente "all'indietro" nel testo: l'istruzione che *usa* il dato appare testualmente prima dell'istruzione che lo *produce* (in una iterazione successiva).
        - **Esempio:**

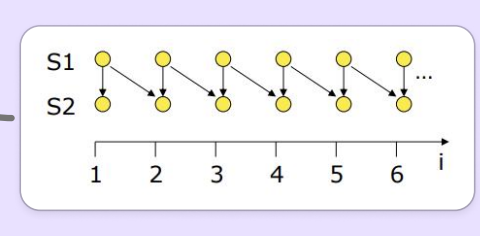
          ```
          C
          for (i=0; i<100; i++) {
              S1: ... = a[i+1];    // Lettura (usa dato futuro)
              S2: a[i] = ...;      // Scrittura
          }
          ```

        - **Analisi:** Qui `S1` all'iterazione `i` legge `a[i+1]`, che verrà scritto da `S2` solo nell'iterazione successiva (`i+1`).
        - **Implicazione:** Questo tipo di dipendenza è spesso una **anti-dipendenza** (WAR). Se è così, può essere talvolta risolta rinominando le variabili o copiando i dati, permettendo comunque una certa parallelizzazione.
    - **dependence with a distance**
      - distanza = numero di iterazioni che separano la generazione del dato del suo utilizzo se un'istruzione all'iterazione i dipende da un'istruzione all'iterazione i-k, allora k è la distanza (o offset)

        ```
        for (i=0; i<100; i++)
          a[i+10] = f(a[i]);
        ```

        - in this case we have, for example, that iteration 0 reads a[0] and writes a[10] while iteratio 10 reads a[10]
          - iteration 10 depends on the result of iteration 0
          - there is a dependency with a **distance of 10**
          - due casi possibili:
            - caso k=1 → strict flow dependence → no parallelismo
            - caso k>1 → possibilità parallelismo parziale
      - **parallelism**
        - we can execute iterations 0 through 9 in parallel because non of them depend on each other
          - then you can execute 10 through 19, and so on
        - we have a batched parallelism of size 10
    - **dependencies *within* the body loop**
      - queste dipendenze non impediscono di parallelizzare il ciclo intero (DOALL) → ogni iterazione è un blocco autonomo → tuttavia non possiamo ordinare gli statements all'interno del singolo thread

        ```
        for (i=0; i<100; i++)
          a[i] = ...;
          ... = a[i-1];
        ```

        - there is a dependency between a[i] (produced in the current iteration at S1) and a[i-1] (consumed in the *next* iteration at S2)
          - specifically, iteration i's S2 depends on iteration i-1's S1
      - **parallelism**
        - statements can be executed in "pipeline" manner
          - as soon as iteration 1 finishes S1 (producing a[1]), iteration 2 can essentially starts its S2 (consuming a[1])
            - we can use pipeline manner to achieve partial parallelism
          - 

            - while S1 of one iteration must precede S2 of the next, multiple S1 operations or multiple S2 operations might be overlapping in time if managed correctly
            - **e.g.,** S1 for i=2 and S2 for i=1 might run concurrently if there are no other conflicts
    - **advanced loop analysis and transformations**
      - **nested loops**

        ```
        for (i=0; i<100; i++)
          for (j=1; j<100; j++)
            a[i][j] = f(a[i][j-1]);
        ```

        - in the **inner loop (j)** a[i][j] depends on a[i][j-1]
          - this is a **loop-carried dependence** on j
          - we cannot calculate j=2 until j=1 is done
          - the inner loop is serial (not parallelizable)
        - in the **outer loop (i)** the calculation for row i does not depend on row i-1 or i+1
          - each row is completely independent
          - this is a **loop-independent dependence** regarding i
          - the outer loop is parallelizable
      - **swaping nested loops**

        ```
        for (j=1; j<100; j++)
          for (i=0; i<100; i++)
            a[i][j] = f(a[i][j-1]);
        ```

        - in the **inner loop (i)** a[i][j] depends on a[i][j-1]
          - since j is fixed for the entire inner loop, a[i][j-1] is kust a value read from memory
            - it is computed in the *previous* outer loop
            - there is no dependency between i and i-1
          - the inner loop is parallelizable
        - in the **outer loop (j)** iteration j depends on iteration j-1 because of the read
          - the outer loop is not parallelizable
      - **which is better?**
        - with **outer parallel**, we create threads *once* for the rows and each thread does a lot of work (the whole inner loop)
          - this has **low overhead**
        - with **inner parallel**, we execute the outer loop sequentially
          - for *each* step of j, you spawn threads to do the i loop in parallel
          - then you must sync and join them before the next j
          - this creates **high overhead**
            - starting/stopping threads 100 times
        - the outer parallel approach is generally **more desirable** due to lower overhead (coarser granularity)
    - to achieve parallelism when dependencies exist
      - **reordering:** ensure statement order does not matter where possible
      - **removal:** remove name dependencies (anti/output) by modifying the code
      - **synchronization:** for remaining dependencies, we must control the ordering of events
        - we use synchronization mechanisms to maintain a partial ordering of execution, allowing concurrent execution of the independent parts while respecting the constraints of the dependent parts